EXISTS, ANY, ALL, UNION, INTERSECT in MySQL – Short Notes

1. EXISTS

What is EXISTS?

`EXISTS` checks whether a **subquery returns at least one row**.

- Returns `TRUE` if rows exist.
- Returns `FALSE` if no rows exist.

Syntax

```
SELECT column_name
FROM table_name
WHERE EXISTS (
    SELECT column_name
    FROM another_table
);
```

Example

Find customers who have placed orders:

```
SELECT name
FROM customers c
WHERE EXISTS (
    SELECT *
    FROM orders o
    WHERE c.id = o.customer_id
);
```

---

2. ANY

What is ANY?

`ANY` compares a value with **any value returned by a subquery**.

The condition is true if it matches **at least one value**.

Syntax

```
SELECT column_name
FROM table_name
WHERE column_name operator ANY (
    SELECT column_name
    FROM another_table
);
```

Example

Find employees whose salary is greater than any employee in department 10:

```
SELECT name, salary
FROM employees
WHERE salary > ANY (
    SELECT salary
    FROM employees
    WHERE department_id = 10
);
```

Meaning:

Salary > at least one salary from department 10.

---

3. ALL

What is ALL?

`ALL` compares a value with **every value returned by a subquery**.

The condition must be true for all values.

Syntax

```
SELECT column_name
FROM table_name
WHERE column_name operator ALL (
    SELECT column_name
    FROM another_table
);
```

Example

Find employees whose salary is greater than all employees in department 10:

```
SELECT name, salary
FROM employees
WHERE salary > ALL (
    SELECT salary
    FROM employees
    WHERE department_id = 10
);
```

Meaning:

Salary must be greater than the highest salary in department 10.

ANY vs ALL

ANY	ALL
Condition must be true for at least one value	Condition must be true for every value
Similar to OR	Similar to AND
Less restrictive	More restrictive

Example:

```
Salary > ANY (30000, 50000, 70000)

Means:
Salary > 30000 OR 50000 OR 70000
```

```
Salary > ALL (30000, 50000, 70000)

Means:
Salary > 70000
```

---

4. UNION

What is UNION?

`UNION` combines the results of two or more `SELECT` queries into one result.

- Removes duplicate rows automatically.
- Number and datatype of columns must match.

Syntax

```sql
SELECT column1 FROM table1

UNION

SELECT column1 FROM table2;
```

Example

```sql
SELECT city
FROM customers

UNION

SELECT city
FROM suppliers;
```

Result:

- All unique cities from both tables.

UNION ALL

`UNION ALL` keeps duplicate rows.

Example:

```
SELECT city FROM customers

UNION ALL

SELECT city FROM suppliers;
```

Difference:

UNION	UNION ALL
Removes duplicates	Keeps duplicates
Slower	Faster

---

5. INTERSECT

What is INTERSECT?

`INTERSECT` returns only the rows that are **common in both SELECT queries**.

Syntax

```
SELECT column1
FROM table1

INTERSECT
```

```sql
SELECT column1
FROM table2;
```

Example

```sql
SELECT city
FROM customers

INTERSECT

SELECT city
FROM suppliers;
```

Result:

- Cities that exist in both tables.

Note about MySQL

Traditional MySQL versions do **not support** `INTERSECT` **directly** (before MySQL 8.0.31).

It can be achieved using `INNER JOIN` or `EXISTS`.

Example using `INNER JOIN`:

```sql
SELECT c.city
FROM customers c
INNER JOIN suppliers s
ON c.city = s.city;
```

UNION vs INTERSECT

UNION	INTERSECT
Combines results from both queries	Returns common results
Works like OR	Works like AND
Removes duplicates	Keeps only matching rows

Quick Revision

Keyword	Purpose
`EXISTS`	Checks if a subquery returns any rows
`ANY`	True if condition matches at least one value
`ALL`	True if condition matches every value
`UNION`	Combines two query results and removes duplicates
`UNION ALL`	Combines results and keeps duplicates
`INTERSECT`	Returns common rows from two queries

Remember:

- `EXISTS` → "Does something exist?"
- `ANY` → "Does it match at least one?"
- `ALL` → "Does it match every one?"
- `UNION` → "Combine everything"
- `INTERSECT` → "Find common data"